

ARTIFICIAL INTELLIGENCE

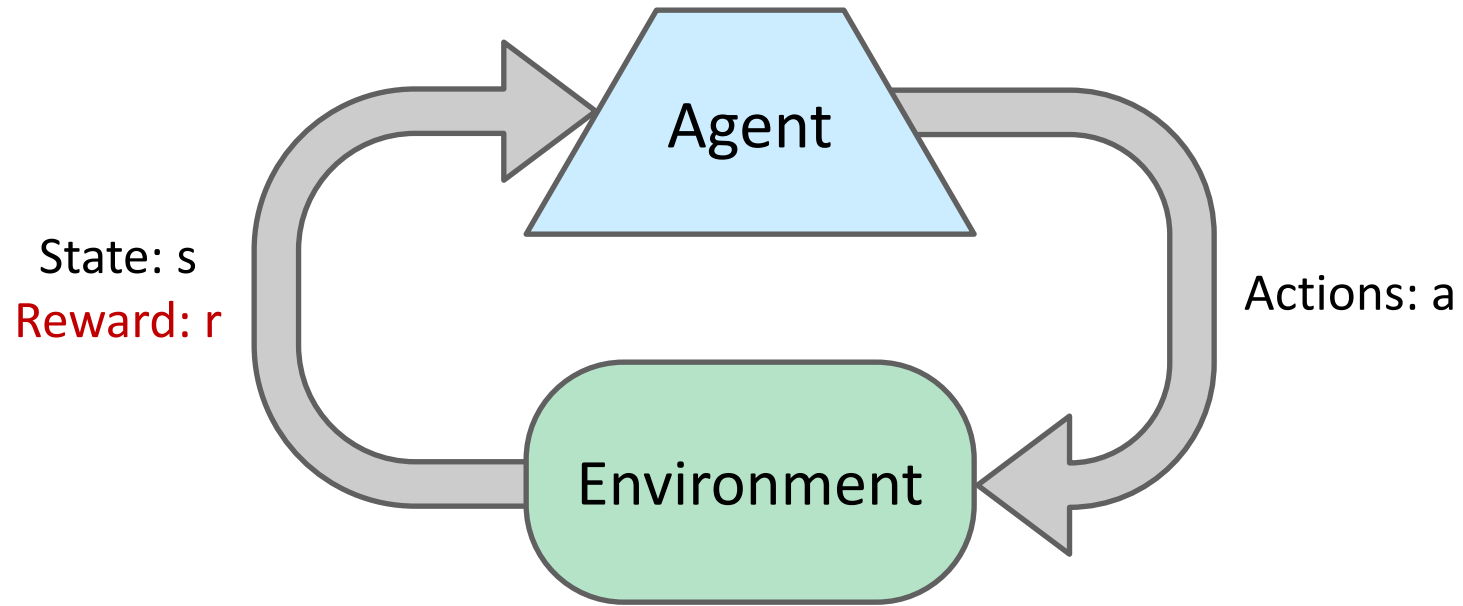
Lecture Set 20
Reinforcement Learning

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

1. Introduction to AI
2. Agents and Environments
3. Problem Solving by Searching
4. Uninformed Search
5. Informed Search
6. Local Search
7. Constraint Satisfaction Problems
8. Adversarial Search
9. Knowledge Representation and Logical Agents
10. Uncertainty, Conditional Probability, Bayes Rule
11. Bayesian Networks, Naïve Bayes Classifier
12. Hidden Markov model, Markov Decision Process
- ➔ **13. Machine Learning (Supervised, Unsupervised, Reinforcement Learning)**

Reinforcement Learning



- **Basic idea:**

- Receive feedback in the form of **rewards**
- Agent's utility is defined by the reward function
- Must (learn to) act so as to **maximize expected rewards**
- All learning is based on observed samples of outcomes!

Reinforcement Learning

- Still assume a Markov decision process (MDP):
 - A set of states $s \in S$
 - A set of actions (per state) A
 - A model $T(s,a,s')$
 - A reward function $R(s,a,s')$
- Still looking for a policy $\pi(s)$
- New twist: don't know T or R
 - I.e. we don't know which states are good or what the actions do
 - Must actually try out actions and states to learn



Offline (MDPs) vs. Online (RL)



Markov decision process (offline)

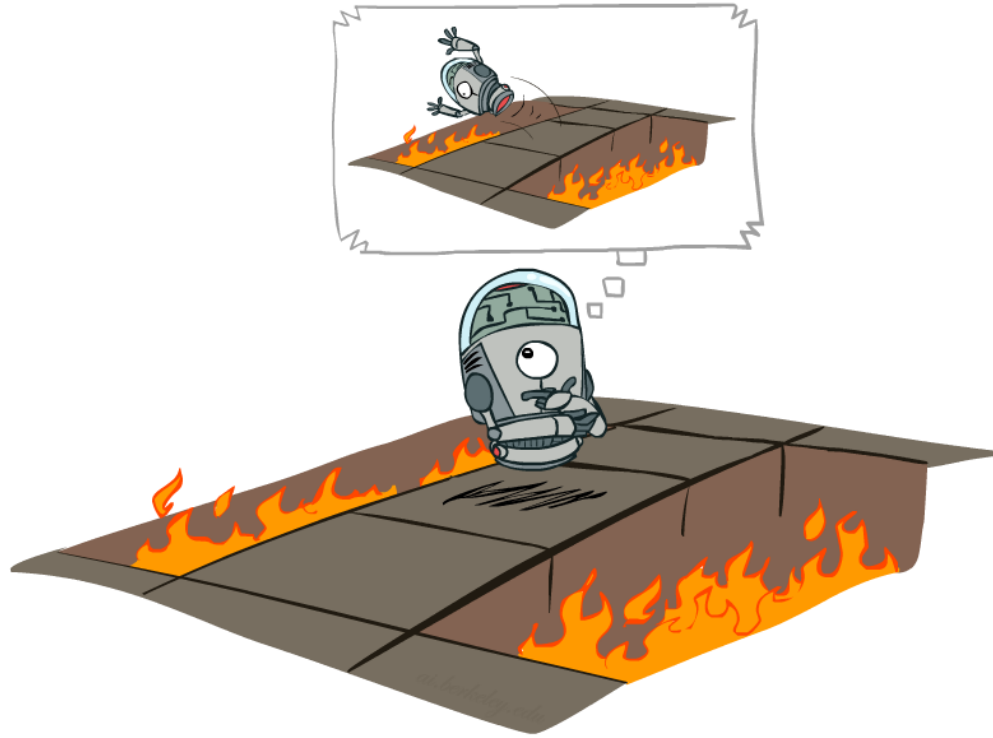
- Have mental model of how the world works.
- Find policy to collect maximum rewards.



Reinforcement learning (online)

- Don't know how the world works.
- Perform actions in the world to find out and collect rewards.

Offline (MDPs) vs. Online (RL)

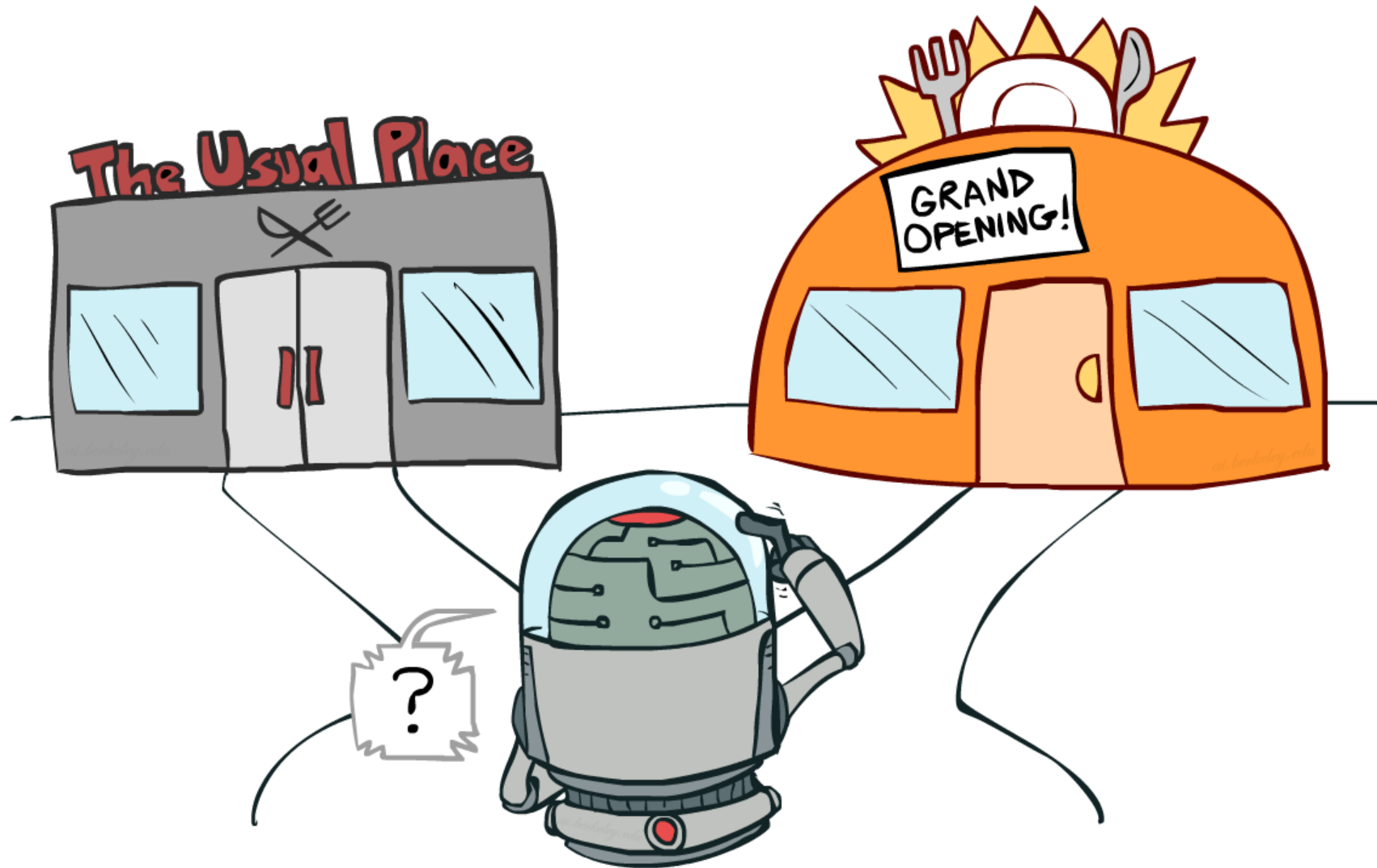


Offline Solution



Online Learning

Exploration vs. Exploitation



How to Explore?

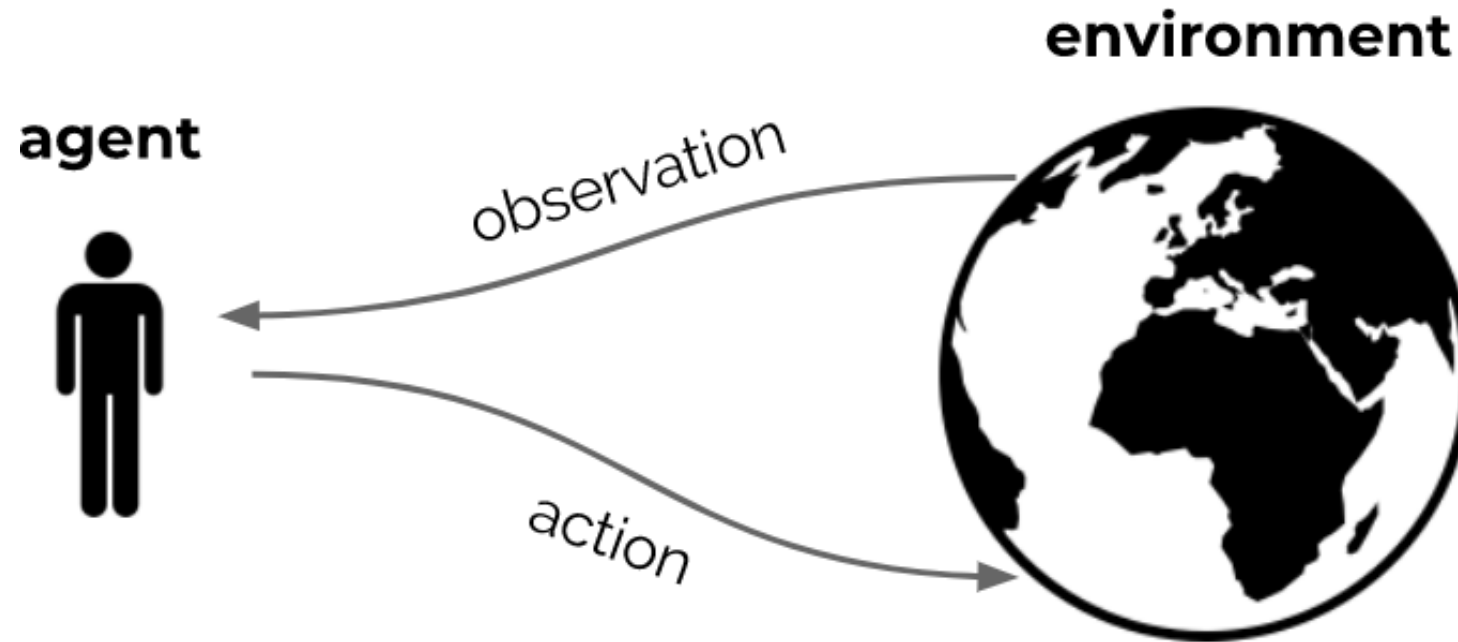
- Several schemes for forcing exploration
 - Simplest: random actions (ϵ -greedy)
 - Every time step, flip a coin
 - With (small) probability ϵ , act randomly
 - With (large) probability $1-\epsilon$, act on current policy
 - Problems with random actions?
 - You do eventually explore the space, but keep thrashing around once learning is done
 - One solution: lower ϵ over time
 - Another solution: exploration functions



What is reinforcement learning?

- ▶ People and animals learn by **interacting with our environment**
- ▶ This differs from certain other types of learning
 - ▶ It is **active** rather than passive
 - ▶ Interactions are often **sequential** — future interactions can depend on earlier ones
- ▶ We are **goal-directed**
- ▶ We can learn **without examples** of optimal behaviour
- ▶ Instead, we optimise some **reward signal**

The interaction loop



Goal: optimise sum of rewards, through repeated interaction

The reward hypothesis

Reinforcement learning is based on the **reward hypothesis**:

Any goal can be formalized as the outcome of maximizing a cumulative reward

Examples of RL problems

- ▶ Fly a helicopter → **Reward**: air time, inverse distance, ...
- ▶ Manage an investment portfolio → **Reward**: gains, gains minus risk, ...
- ▶ Control a power station → **Reward**: efficiency, ...
- ▶ Make a robot walk → **Reward**: distance, speed, ...
- ▶ Play video or board games → **Reward**: win, maximise score, ...

If the goal is to learn via interaction, these are all reinforcement learning problems (Irrespective of which solution you use)

What is reinforcement learning?

There are distinct reasons to learn:

1. Find **solutions**

- ▶ A program that plays chess really well
- ▶ A manufacturing robot with a specific purpose

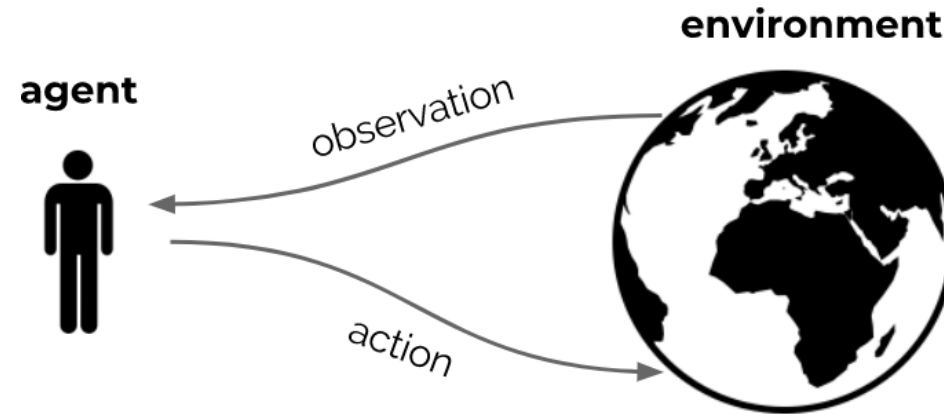
2. **Adapt online**, deal with unforeseen circumstances

- ▶ A chess program that can learn to adapt to you
 - ▶ A robot that can learn to navigate unknown terrains
-
- ▶ Reinforcement learning can provide algorithms for both cases
 - ▶ Note that the second point is not (just) about generalization — it is about continuing to learn efficiently online, during operation

What is reinforcement learning?

- ▶ Science and framework of **learning to make decisions** from **interaction**
- ▶ This requires us to think about
 - ▶ ...time
 - ▶ ...(long-term) consequences of actions
 - ▶ ...actively gathering experience
 - ▶ ...predicting the future
 - ▶ ...dealing with uncertainty
- ▶ Huge potential scope
- ▶ A formalisation of the AI problem

Agent and Environment



- ▶ At each step t the agent:
 - ▶ Receives observation O_t (and reward R_t)
 - ▶ Executes action A_t
- ▶ The environment:
 - ▶ Receives action A_t
 - ▶ Emits observation O_{t+1} (and reward R_{t+1})

Rewards

- ▶ A **reward** R_t is a scalar feedback signal
- ▶ Indicates how well agent is doing at step t — defines the goal
- ▶ The agent's job is to maximize cumulative reward

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

- ▶ We call this the **return**

Reinforcement learning is based on the **reward hypothesis**:

Any goal can be formalized as the outcome of maximizing a cumulative reward

Values

- ▶ We call the expected cumulative reward, from a state s , the **value**

$$\begin{aligned}v(s) &= \mathbb{E} [G_t \mid S_t = s] \\&= \mathbb{E} [R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s]\end{aligned}$$

- ▶ The value depends on the actions the agent takes
- ▶ Goal is to **maximize value**, by picking suitable actions
- ▶ Rewards and values define **utility** of states and action (no supervised feedback)
- ▶ Returns and values can be defined recursively

$$\begin{aligned}G_t &= R_{t+1} + G_{t+1} \\v(s) &= \mathbb{E} [R_{t+1} + v(S_{t+1}) \mid S_t = s]\end{aligned}$$

Maximising value by taking actions

- ▶ Goal: **select actions to maximise value**
- ▶ Actions may have long term consequences
- ▶ Reward may be delayed
- ▶ It may be better to sacrifice immediate reward to gain more long-term reward
- ▶ Examples:
 - ▶ Refueling a helicopter (might prevent a crash in several hours)
 - ▶ Defensive moves in a game (may help chances of winning later)
 - ▶ Learning a new skill (can be costly & time-consuming at first)
- ▶ A mapping from states to actions is called a **policy**

Action values

- ▶ It is also possible to condition the value on **actions**:

$$\begin{aligned} q(s, a) &= \mathbb{E} [G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E} [R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s, A_t = a] \end{aligned}$$

- ▶ We will talk in depth about state and action values later

Core concepts

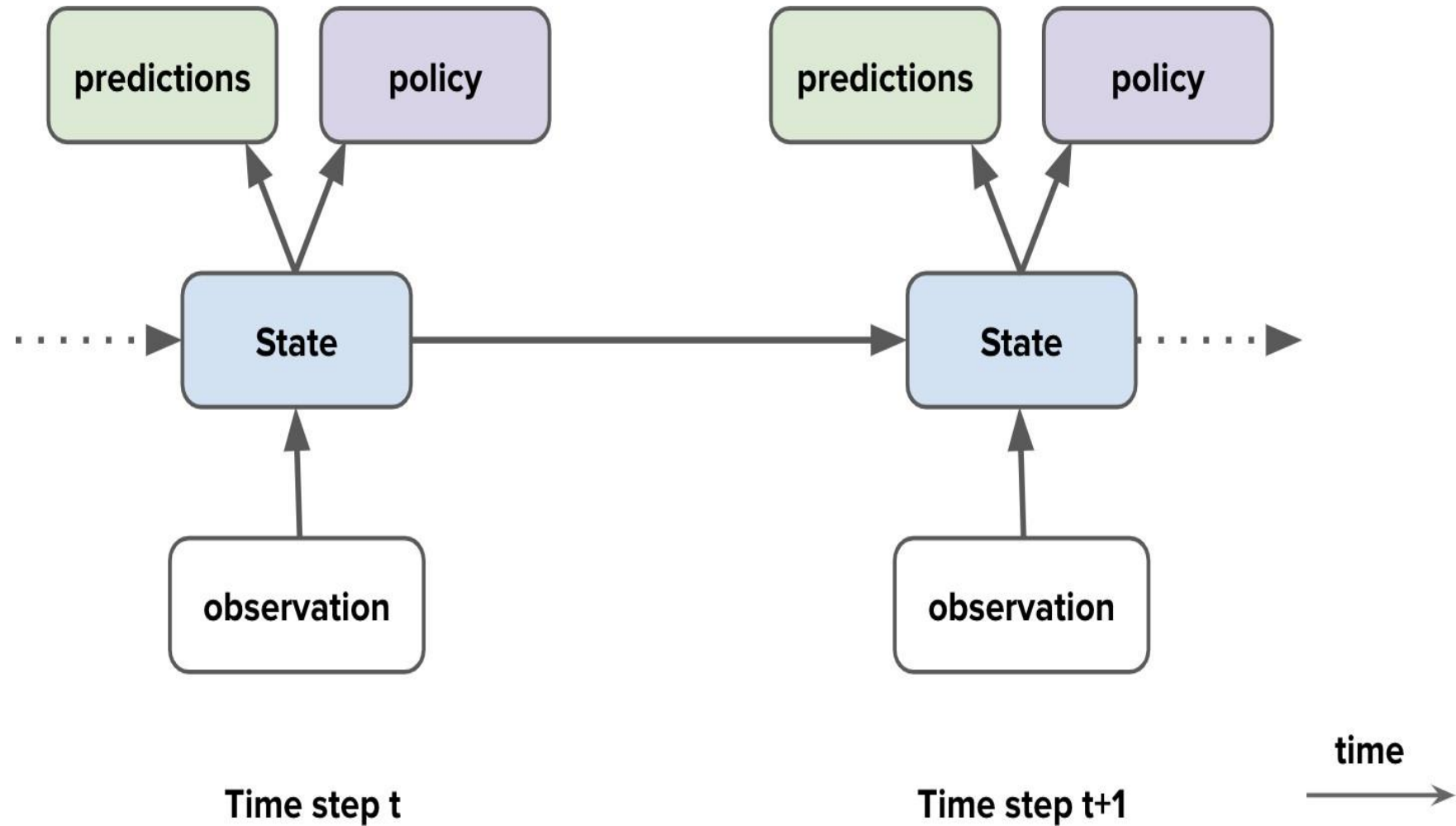
The reinforcement learning formalism includes

- ▶ **Environment** (dynamics of the problem)
- ▶ **Reward** signal (specifies the goal)
- ▶ **Agent**, containing:
 - ▶ Agent state
 - ▶ Policy
 - ▶ Value function estimate?
 - ▶ Model?
- ▶ We will now go into the agent

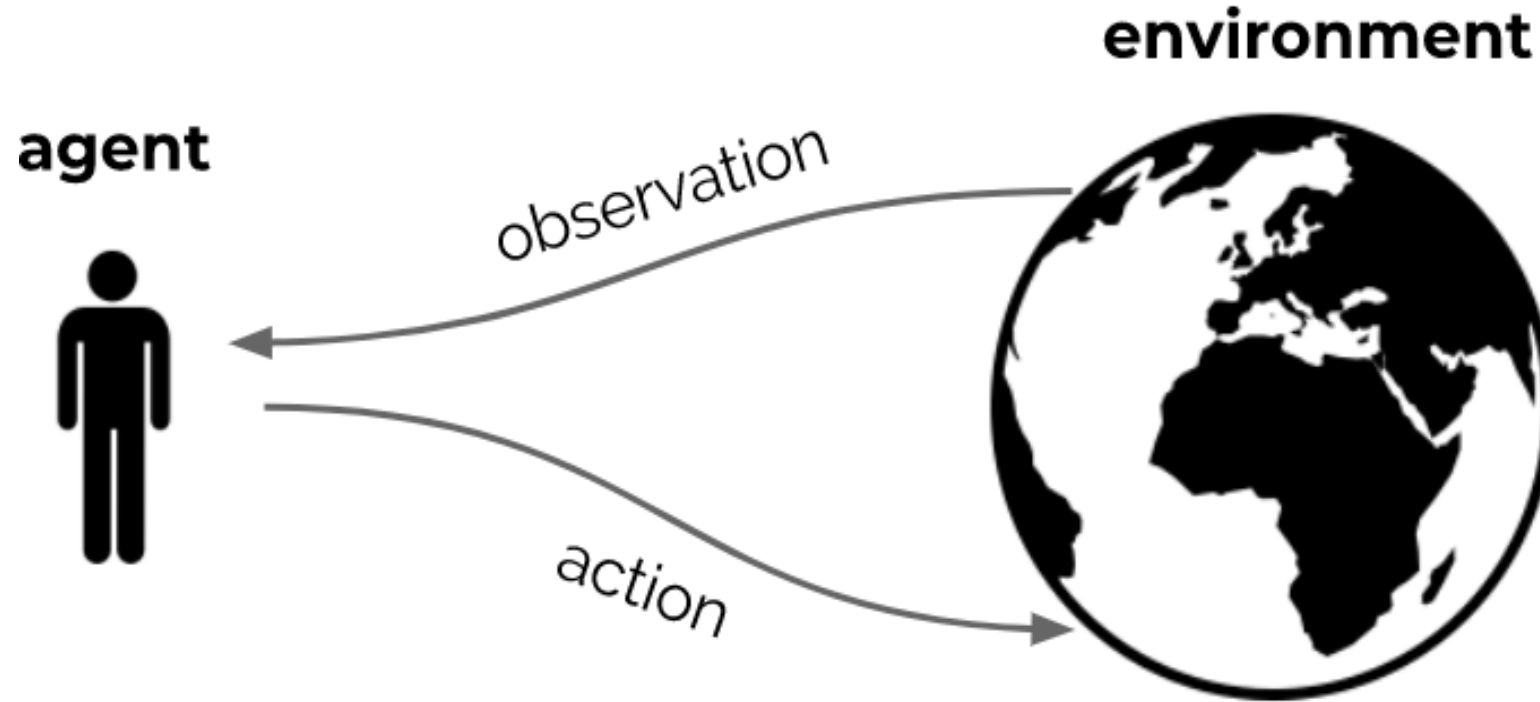
Agent components

Agent components

- ▶ **Agent state**
- ▶ Policy
- ▶ Value functions
- ▶ Model



Environment State



- ▶ The **environment state** is the environment's internal state
- ▶ It is usually invisible to the agent
- ▶ Even if it is visible, it may contain lots of irrelevant information

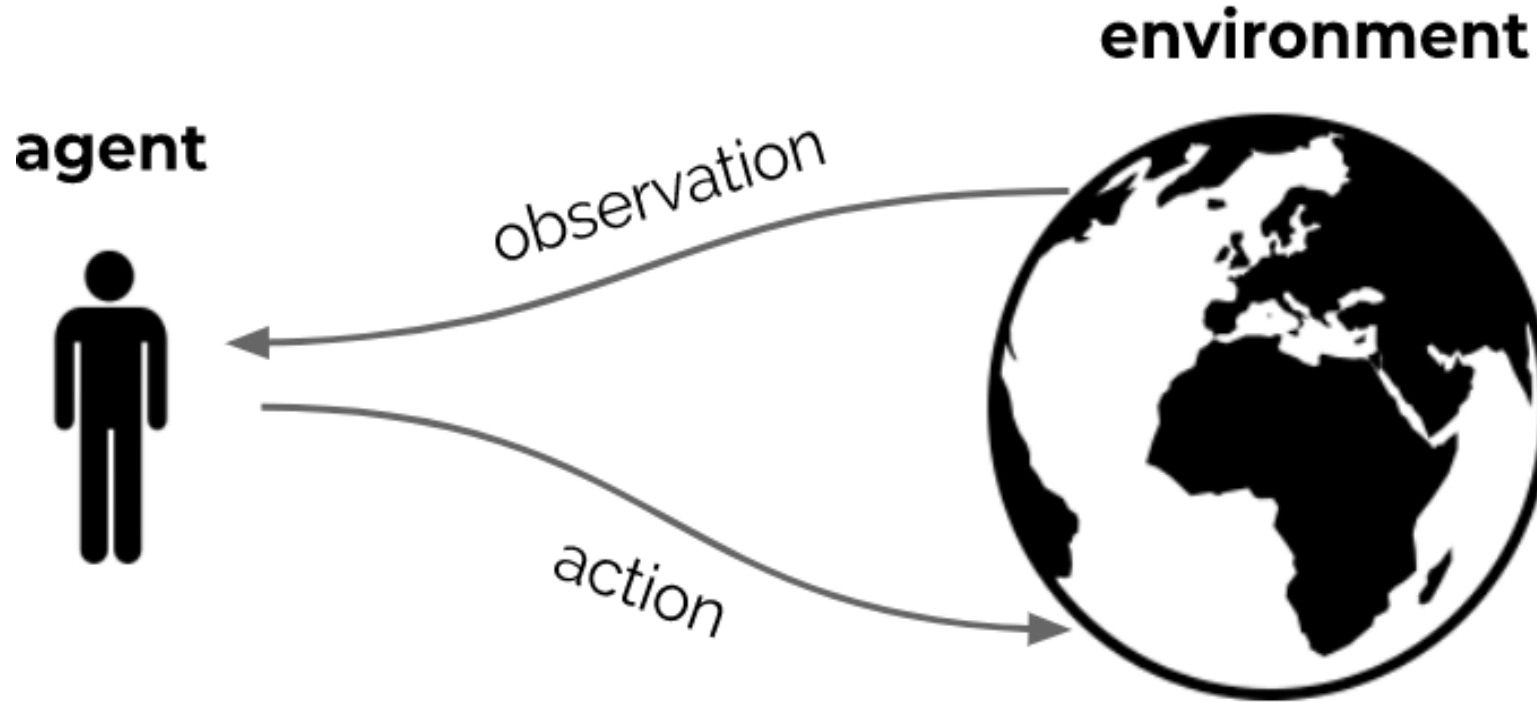
Agent State

- ▶ The **history** is the full sequence of observations, actions, rewards

$$\mathcal{H}_t = O_0, A_0, R_1, O_1, \dots, O_{t-1}, A_{t-1}, R_t, O_t$$

- ▶ For instance, the sensorimotor stream of a robot
- ▶ This history is used to construct the **agent state** S_t

Fully Observable Environments



Full observability

Suppose the agent sees the full environment state

- ▶ observation = environment state
- ▶ The agent state could just be this observation:

$$S_t = O_t = \text{environment state}$$

Markov decision processes

Markov decision processes (MDPs) are a useful mathematical framework

Definition

A decision process is Markov if

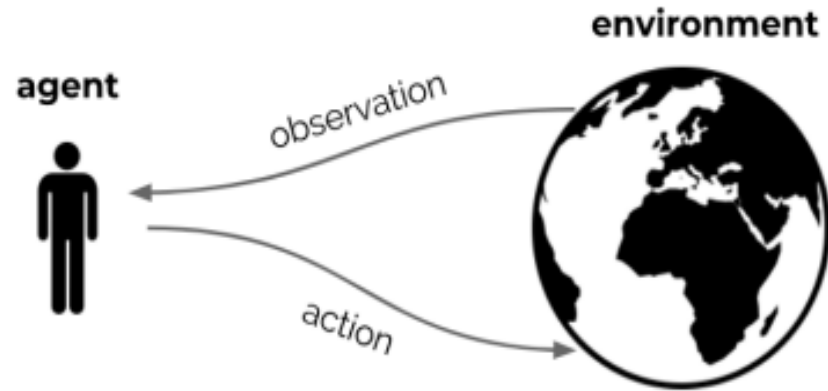
$$p(r, s \mid S_t, A_t) = p(r, s \mid \mathcal{H}_t, A_t)$$

- ▶ This means that the state contains all we need to know from the history
- ▶ Doesn't mean it contains everything, just that adding more history doesn't help
- ▶ $\implies$ Once the state is known, the history may be thrown away
 - ▶ The full environment + agent state is Markov (but large)
 - ▶ The full history $\mathcal{H}_t$ is Markov (but keeps growing)
- ▶ Typically, the agent state S_t is some compression of $\mathcal{H}_t$
- ▶ Note: we use S_t to denote the **agent state**, not the **environment state**

Partially Observable Environments

- ▶ **Partial observability**: The observations are not Markovian
 - ▶ A robot with camera vision isn't told its absolute location
 - ▶ A poker playing agent only observes public cards
- ▶ Now using the observation as state would not be Markovian
- ▶ This is called a **partially observable Markov decision process** (POMDP)
- ▶ The **environment state** can still be Markov, but the agent does not know it
- ▶ We might still be able to construct a Markov agent state

Agent State



- ▶ The agent's actions depend on its state
- ▶ The **agent state** is a function of the history
- ▶ For instance, $S_t = O_t$
- ▶ More generally:

$$S_{t+1} = u(S_t, A_t, R_{t+1}, O_{t+1})$$

where u is a 'state update function'

- ▶ The agent state is often **much** smaller than the environment state

Agent State

The full environment state of a maze



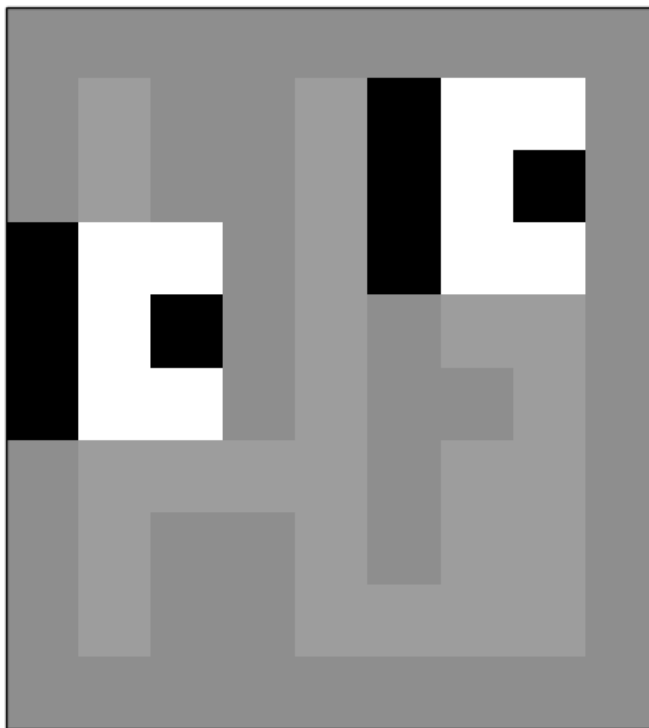
Agent State

A potential observation



Agent State

The two observations are indistinguishable



Agent State

These two states are not Markov



How could you construct a Markov agent state in this maze (for any reward signal)?

Partially Observable Environments

- ▶ To deal with partial observability, agent can construct suitable state representations
- ▶ Examples of agent states:
 - ▶ Last observation: $S_t = O_t$ (might not be enough)
 - ▶ Complete history: $S_t = \mathcal{H}_t$ (might be too large)
 - ▶ A generic update: $S_t = \nu(S_{t-1}, A_{t-1}, R_t, O_t)$ (but how to pick/learn ν ?)
- ▶ Constructing a fully Markovian agent state is often not feasible
- ▶ More importantly, the state should allow good policies and value predictions

Agent components

Agent components

- ▶ Agent state
- ▶ **Policy**
- ▶ Value function
- ▶ Model

Policy

- ▶ A **policy** defines the agent's behaviour
- ▶ It is a map from agent state to action
- ▶ Deterministic policy: $A = \pi(S)$
- ▶ Stochastic policy: $\pi(A|S) = p(A|S)$

Agent components

Agent components

- ▶ Agent state
- ▶ Policy
- ▶ **Value function**
- ▶ Model

Value Function

- ▶ The actual value function is the expected return

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}[G_t \mid S_t = s, \pi] \\&= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s, \pi]\end{aligned}$$

- ▶ We introduced a **discount factor** $\gamma \in [0, 1]$
 - ▶ Trades off importance of immediate vs long-term rewards
- ▶ The value depends on a policy
- ▶ Can be used to evaluate the desirability of states
- ▶ Can be used to select between actions

Value Functions

- ▶ The return has a recursive form $G_t = R_{t+1} + \gamma G_{t+1}$
- ▶ Therefore, the value has as well

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E} [R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t \sim \pi(s)] \\ &= \mathbb{E} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t \sim \pi(s)] \end{aligned}$$

Here $a \sim \pi(s)$ means a is chosen by policy π in state s (even if π is deterministic)

- ▶ This is known as a **Bellman equation** (Bellman 1957)
- ▶ A similar equation holds for the optimal (=highest possible) value:

$$v_*(s) = \max_a \mathbb{E} [R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

This does **not** depend on a policy

- ▶ We heavily exploit such equalities, and use them to create algorithms

Value Function approximations

- ▶ Agents often approximate value functions
- ▶ We will discuss algorithms to learn these efficiently
- ▶ With an accurate value function, we can behave optimally
- ▶ With suitable approximations, we can behave well, even in intractably big domains

Agent components

Agent components

- ▶ Agent state
- ▶ Policy
- ▶ Value function
- ▶ **Model**

Model

- ▶ A **model** predicts what the environment will do next
- ▶ E.g., $\mathcal{P}$ predicts the next state

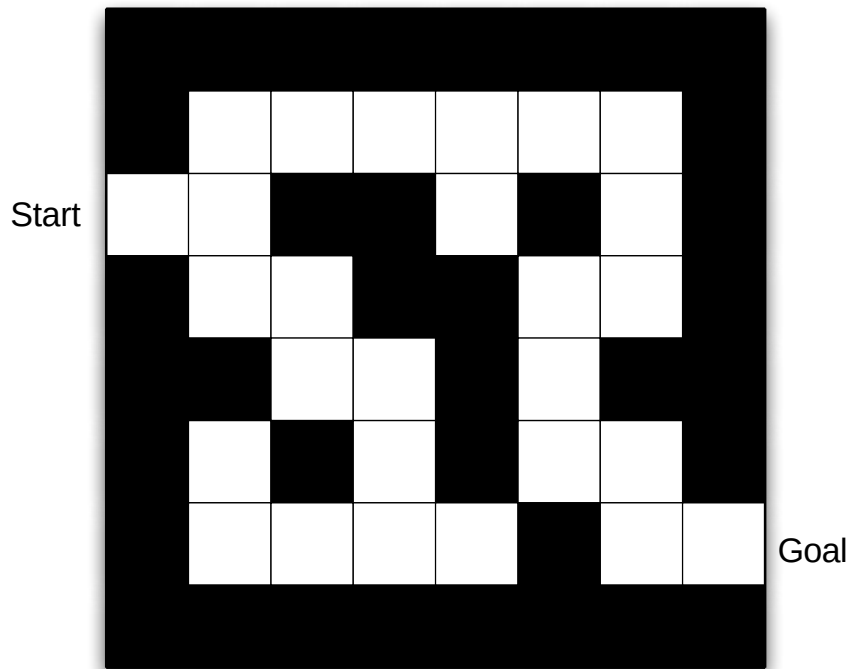
$$\mathcal{P}(s, a, s') \approx p(S_{t+1} = s' \mid S_t = s, A_t = a)$$

- ▶ E.g., $\mathcal{R}$ predicts the next (immediate) reward

$$\mathcal{R}(s, a) \approx \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

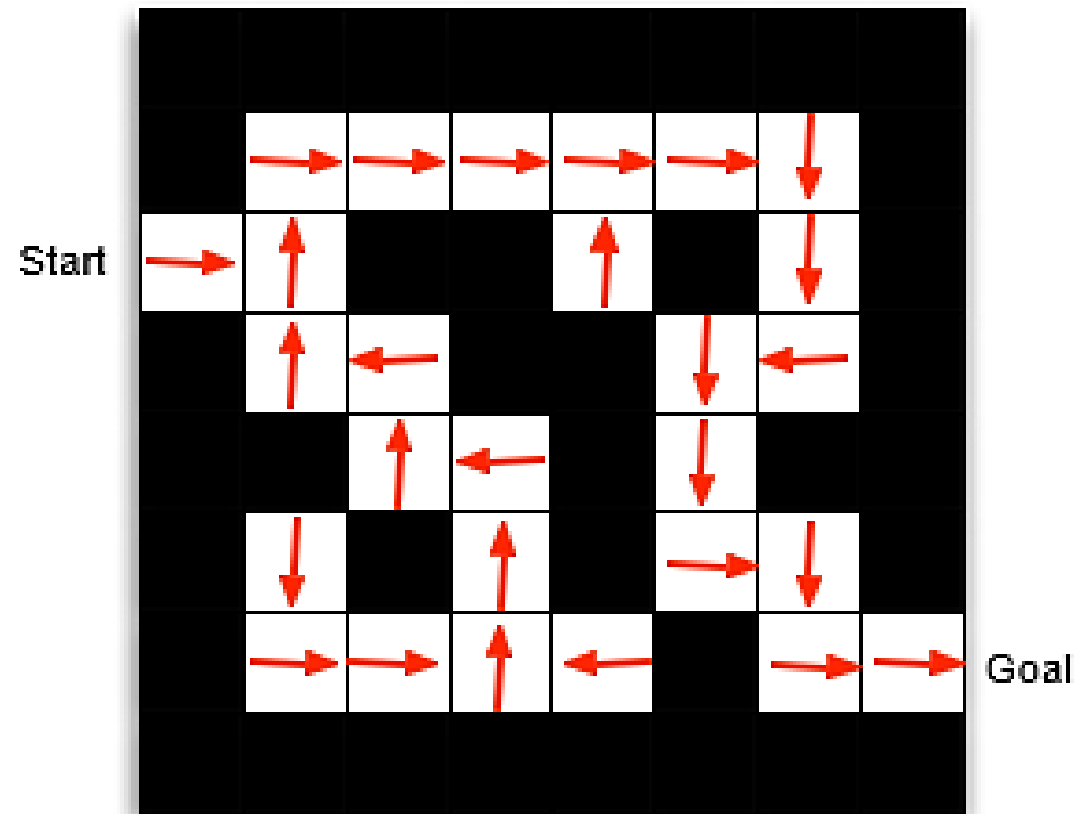
- ▶ A model does not immediately give us a good policy - we would still need to plan
- ▶ We could also consider **stochastic (generative)** models

Maze Example



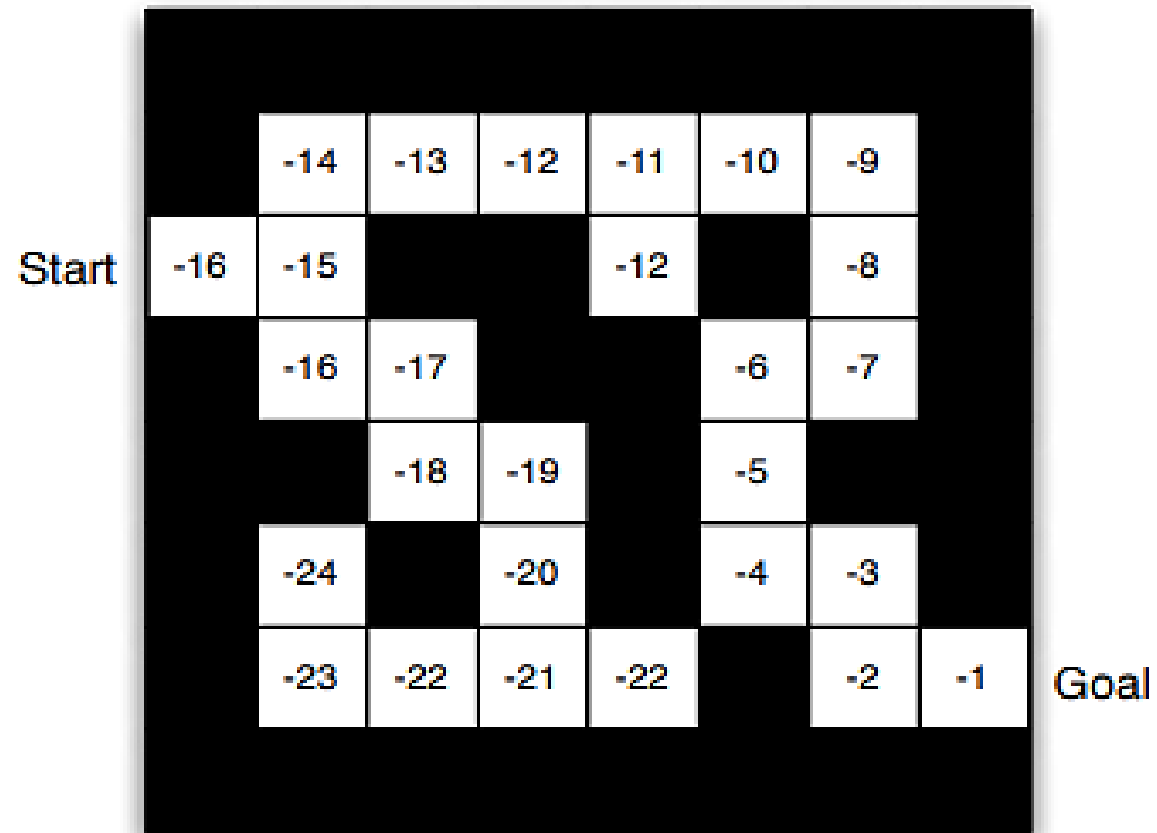
- ▶ Rewards: -1 per time-step
- ▶ Actions: N, E, S, W
- ▶ States: Agent's location

Maze Example: Policy



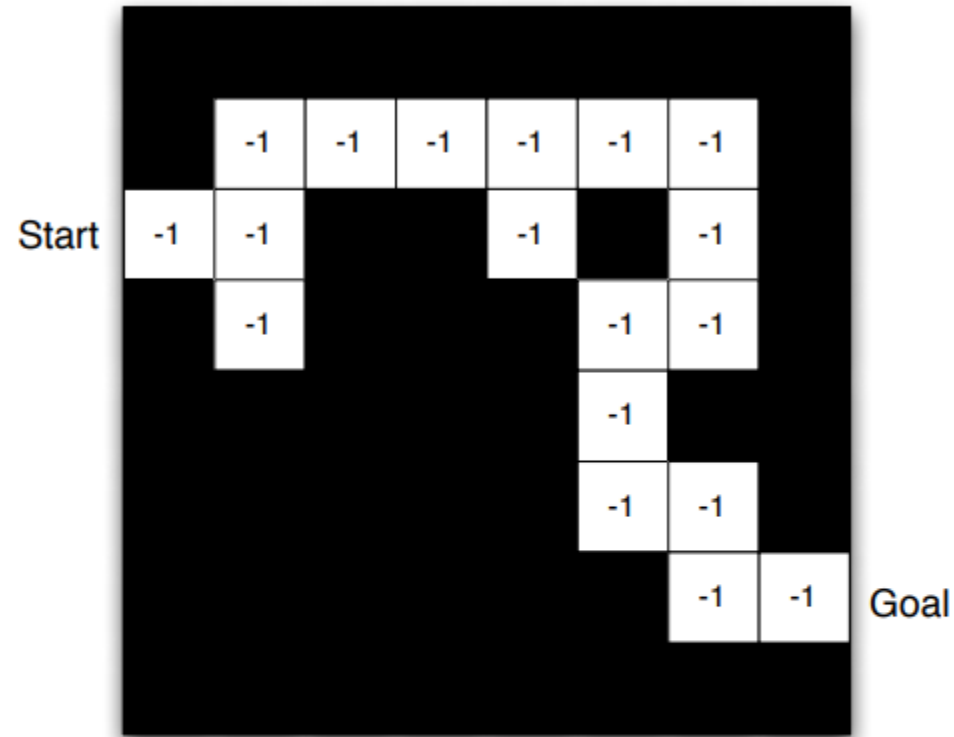
- Arrows represent policy $\pi(s)$ for each state s

Maze Example: Value Function



- Numbers represent value $v_{\pi}(s)$ of each state s

Maze Example: Model



- ▶ Grid layout represents partial transition model $\mathcal{P}_{ss'}^a$
- ▶ Numbers represent immediate reward $\mathcal{R}_{ss'}^a$ from each state s (same for all a and s' in this case)